

Lab04 - PHP Secure Forms, PRG, Anti-spam & Session Login Flow

Câu 1. Thực hiện lại Lab04 trên một ứng dụng mới

Yêu cầu tổng quát: Hãy thực hiện lại toàn bộ theo bài Lab04 hướng dẫn, nhưng áp dụng vào một bài toán khác có mức độ tương đương với Student Consultation & Secure Login Portal. Mục tiêu không phải copy y nguyên bài mẫu, mà là hiểu và triển khai lại đúng flow bảo mật form/session.

1.1. Sinh viên được chọn một trong các hướng bài toán

- Mini Course Consultation Portal - cổng đăng ký tư vấn khóa học.
- Mini Clinic Appointment Portal - cổng đặt lịch khám/tư vấn phòng khám.
- Mini Workshop Registration Portal - cổng đăng ký tham gia workshop.
- Mini Equipment Borrow Request Portal - cổng đăng ký mượn thiết bị.
- Mini Student Support Request Portal - cổng gửi yêu cầu hỗ trợ sinh viên.
- Mini Training Center Lead Portal - cổng nhận lead tư vấn trung tâm đào tạo.
- Mini Event Registration Portal - cổng đăng ký sự kiện nhỏ.

Lưu ý: Nếu chọn bài toán khác ngoài danh sách trên, sinh viên phải đảm bảo vẫn có form POST, validation server-side, PRG, anti-spam cơ bản, login/logout/session dashboard và test 404/405.

1.2. Vẫn thực hiện đầy đủ các bước như bài hướng dẫn Lab04

- Kiểm tra môi trường PHP, Composer, Git và mở project bằng VS Code.
- Có thể copy project Lab03 rồi đổi tên sang project Lab04 mới, hoặc tạo project mới có Front Controller và Router.
- Tạo cấu trúc thư mục app/, public/, views/, storage/ rõ ràng.
- Tạo composer.json, cấu hình PSR-4 autoload và chạy composer dump-autoload.
- Tạo public/index.php làm Front Controller, khai báo route tập trung.
- Cấu hình session cookie params trước session_start(): HttpOnly, SameSite, Secure theo môi trường.
- Tạo helper: h()/escape, redirect(), flash_set(), flash_get(), is_logged_in(), require_login(), timeout/logout helpers.
- Tạo Router xử lý đúng 404 Not Found và 405 Method Not Allowed.
- Tạo layout, CSS chung, các view HTML phù hợp với bài toán đã chọn.
- Tạo form đăng ký/yêu cầu/tư vấn và route xử lý POST tương ứng.
- Đọc input an toàn bằng \$_POST, trim dữ liệu và không tin dữ liệu user nhập.
- Xây dựng validation server-side: required, email format, phone pattern, length và in-list validation.
- Hiển thị lỗi đúng field, giữ lại dữ liệu cũ khi submit sai bằng flash old/errors.
- Áp dụng PRG: sau POST thành công phải redirect về một route GET.
- Thêm honeypot field và rate limit bằng session để giảm spam cơ bản.
- Lưu dữ liệu vào file JSON trong storage/ hoặc một storage tương đương. Lab04 không bắt buộc dùng database.
- Tạo login/logout flow: validate input, verify password demo, session_regenerate_id(true), redirect và flash message.
- Tạo dashboard chỉ cho user đã đăng nhập, hiển thị thông tin session phục vụ debug.
- Thực hiện timeout phiên và logout sạch: xóa session data, destroy session và xóa cookie phiên.
- Test bằng browser, DevTools, curl hoặc Postman.
- Quản lý source code bằng Git, commit rõ ràng và đưa project lên GitHub.

1.3. Ứng dụng mới phải có các route tối thiểu

Method	URL	Controller@Action	Response / ý nghĩa
GET	/	HomeController@index	HTML trang tổng quan ứng dụng
GET	/requests hoặc /consultations hoặc /appointments	MainResourceController@index	HTML danh sách dữ liệu + flash success/error
GET	/requests/create hoặc route tương đương	MainResourceController@create	HTML form tạo yêu cầu/lead/appointment
POST	/requests hoặc route tương đương	MainResourceController@store	Validate + anti-spam + lưu JSON + redirect
GET	/login	AuthController@login	HTML form login
POST	/login	AuthController@handleLogin	Validate + verify password + regenerate session + redirect
POST	/logout	AuthController@logout	Logout sạch + redirect về login
GET	/dashboard	DashboardController@index	Chỉ cho user đã đăng nhập
GET	/session-demo	DashboardController@sessionDemo	Hiển thị thông tin session phục vụ debug
ANY	URL không tồn tại	Router	404 Not Found
Sai method	Route có tồn tại nhưng method sai	Router	405 Method Not Allowed

1.4. Sinh viên phải đổi toàn bộ cho phù hợp bài toán mới

- Tên project, tên bài toán, nội dung trang chủ.
- Tên dữ liệu chính, ví dụ appointment, request, registration, equipment_request, support_ticket.
- Tên class, controller, action, route, view, file JSON.
- Tên field trong form và nội dung hiển thị trên giao diện.
- Danh sách giá trị in-list validation, ví dụ loại khóa học, loại lịch hẹn, loại thiết bị, loại hỗ trợ.
- Thông báo lỗi, thông báo thành công, dữ liệu mẫu, thông tin dashboard.
- Commit message, README, ảnh minh họa kết quả chạy.

Bắt buộc: Không được giữ nguyên toàn bộ bài mẫu Student Consultation & Secure Login Portal, ConsultationController, /consultations nếu đã chọn bài toán khác. Có thể giữ tư duy và kỹ thuật, nhưng phải đổi ngữ cảnh nghiệp vụ.

1.5. Yêu cầu kỹ thuật bắt buộc

Nhóm yêu cầu	Mô tả bắt buộc
GET/POST	Dùng GET để hiển thị form/list, dùng POST để gửi dữ liệu hoặc logout. Không dùng GET để tạo dữ liệu.
Input safety	Đọc input bằng \$_POST/\$_GET có ??, trim(), không tin dữ liệu từ user.
Escape output	Mọi dữ liệu người dùng nhập khi hiển thị ra HTML phải qua h()/htmlspecialchars().
Server-side validation	Bắt buộc có required, email format, phone pattern, length và in-list validation.

Nhóm yêu cầu	Mô tả bắt buộc
Field errors + old input	Khi lỗi, hiển thị lỗi đúng field và giữ lại dữ liệu cũ để user sửa.
PRG	POST thành công không render trực tiếp, phải redirect về GET để tránh submit trùng.
Flash message	Success/error chỉ hiện một lần sau redirect, không bị hiện mãi.
Honeypot	Có field bẫy ẩn, nếu có dữ liệu thì chặn request như hành vi bot.
Rate limit	Chặn submit quá nhanh bằng session, ví dụ không cho gửi thành công 2 lần trong 5 giây.
Session cookie flags	Cấu hình HttpOnly, SameSite=Lax, Secure theo môi trường trước session_start().
Login	Login đúng phải gọi session_regenerate_id(true), lưu user_id, role, login_at, last_activity_at.
Dashboard protection	User chưa login vào /dashboard phải bị redirect về /login kèm flash.
Timeout	Có idle timeout. Khi hết hạn phải logout sạch và redirect về login.
Logout sạch	POST /logout phải xóa session data, destroy session và xóa cookie phiên.
Remember me	Chỉ giới thiệu rủi ro. Tuyệt đối không lưu password trong cookie.
404/405	Router phải phân biệt URL không tồn tại và route tồn tại nhưng sai method.

1.6. Ví dụ chuyển đổi bài toán

Nếu chọn Mini Clinic Appointment Portal, hệ thống có thể quản lý: họ tên bệnh nhân, email, số điện thoại, chuyên khoa cần tư vấn, nội dung triệu chứng, trạng thái lịch hẹn. Route gợi ý:

- GET /appointments - danh sách lịch hẹn/tư vấn.
- GET /appointments/create - form đặt lịch.
- POST /appointments - validate + anti-spam + lưu JSON + redirect.
- GET /login, POST /login, POST /logout, GET /dashboard, GET /session-demo.

Câu 1B. Bài kiểm tra nhỏ bắt buộc sau khi làm xong

Mục tiêu: Các bài kiểm tra nhỏ dưới đây giúp chứng minh sinh viên không chỉ copy code mà hiểu được cơ chế của Lab04. Mỗi mục cần có ảnh chụp màn hình hoặc mô tả kết quả trong file nộp.

Mã	Mục kiểm tra	Cách thực hiện	Kết quả cần đạt
T01	GET/POST flow	Mở form bằng GET, submit bằng POST. Kiểm tra route POST không hiển thị form trực tiếp.	URL và method đúng, không tạo dữ liệu bằng GET.
T02	Validation rỗng	Submit form rỗng.	Hiển thị lỗi cạnh field bắt buộc, giữ dữ liệu cũ nếu có.
T03	Validation định dạng	Nhập email sai, phone sai, tên chỉ 1 ký tự, chọn course/service không hợp lệ.	Có lỗi đúng field và không lưu JSON mới.
T04	Old input	Nhập một số field đúng, một số field sai rồi submit.	Field đúng vẫn giữ lại để user sửa, không phải nhập lại từ đầu.
T05	PRG chống refresh submit lại	Submit hợp lệ, ghi lại số dòng JSON, refresh trang danh sách.	Không phát sinh thêm record sau khi refresh GET.

Mã	Mục kiểm tra	Cách thực hiện	Kết quả cần đạt
T06	Flash message	Sau submit thành công, refresh lại trang danh sách.	Flash success hiện một lần, refresh lần sau không còn hiện lại.
T07	Honeypot	Dùng DevTools/Postman gửi thêm field website có dữ liệu.	Request bị chặn, có global error hoặc không lưu record mới.
T08	Rate limit	Gửi thành công 2 lần liên tiếp trong dưới 5 giây.	Lần gửi quá nhanh bị chặn và không lưu record mới.
T09	Escape output	Nhập thử nội dung dạng <code><script>alert(1)</script></code> .	Nội dung hiển thị ra dạng text an toàn, không chạy script.
T10	Login sai	Nhập sai email/password demo.	Không vào dashboard, có flash error rõ ràng.
T11	Login đúng + regenerate	Login đúng rồi vào /session-demo hoặc dashboard.	Có trạng thái logged in; giải thích được session ID cần đổi sau login.
T12	Dashboard protection	Mở /dashboard khi chưa đăng nhập.	Bị redirect về /login kèm thông báo yêu cầu đăng nhập.
T13	Logout sạch	Đang login, bấm logout bằng POST rồi mở lại /dashboard.	Không còn truy cập được dashboard; bị redirect về login.
T14	GET /logout	Thử truy cập /logout bằng GET nếu hệ thống chỉ khai báo POST /logout.	Không logout bằng GET; route trả 405 hoặc không tồn tại tùy thiết kế.
T15	Timeout	Đặt idle timeout demo ngắn, chờ quá thời gian rồi truy cập dashboard.	Phiên hết hạn, logout sạch, redirect về login với flash.
T16	404/405	Mở URL sai và gọi sai method cho route có thật.	Phân biệt được 404 Not Found và 405 Method Not Allowed.

Câu 2. Problem Solving

Hãy trả lời các câu hỏi **Problem Solving** sau dựa trên chính ứng dụng em đã xây dựng ở Câu 1. Không trả lời chung chung. Mỗi câu nên có ví dụ gắn với route, form, session hoặc lỗi trong bài của em.

1. Vì sao server-side validation là bắt buộc?

- Trong bài của em, client-side validation có đủ để bảo vệ dữ liệu không?
- Nếu user dùng Postman/curl gửi request bỏ qua HTML required thì server xử lý thế nào?
- Hãy lấy ví dụ một field trong form của em và rule validation tương ứng.

2. Vì sao phải phân biệt GET và POST?

- Trong bài của em, route nào dùng GET và route nào dùng POST?
- Vì sao form tạo dữ liệu không nên dùng GET?
- Vì sao logout nên dùng POST thay vì GET?

3. PRG giải quyết vấn đề gì?

- Nếu POST xong render kết quả trực tiếp thì refresh trình duyệt có thể gây lỗi gì?
- Trong bài của em, sau POST thành công redirect về route GET nào?
- Em chứng minh thế nào rằng refresh không tạo dữ liệu trùng?

4. Validation và anti-spam khác nhau như thế nào?

- Validation kiểm tra “dữ liệu đúng” còn anti-spam kiểm tra “hành vi đáng ngờ” ra sao?

- Một bot gửi đủ tên/email/phone hợp lệ thì validation có chặn được không?
- Honeypot và rate limit trong bài em giúp giảm rủi ro nào?

5. Honeypot có giới hạn gì?

- Vì sao honeypot chỉ chặn được bot đơn giản?
- Bot tinh vi có thể bỏ qua honeypot bằng cách nào?
- Nếu hệ thống chạy thật, em đề xuất thêm lớp bảo vệ nào?

6. Rate limit bằng session có ưu và nhược điểm gì?

- Rate limit bằng session có chặn được user đổi trình duyệt hoặc xóa cookie không?
- Nếu muốn mạnh hơn, nên rate limit theo IP, user, email hay server-side store?
- Trong bài của em, giới hạn mấy giây là hợp lý cho demo?

7. Vì sao session_set_cookie_params() phải chạy trước session_start()?

- Nếu gọi session_start() trước rồi mới set cookie params thì có vấn đề gì?
- Hãy giải thích ý nghĩa của HttpOnly, SameSite và Secure.
- Ở môi trường local HTTP, Secure nên bật hay tắt? Vì sao?

8. Vì sao login thành công phải session_regenerate_id(true)?

- Session fixation là gì ở mức dễ hiểu?
- Trước login và sau login, mức độ tin cậy của session khác nhau thế nào?
- Trong bài của em, đoạn code regenerate nằm ở đâu?

9. Logout sạch nghĩa là gì?

- Chỉ unset(\$_SESSION["user_id"]) đã đủ chưa?
- logout_clean() trong bài của em cần xóa những gì?
- Sau logout, mở lại /dashboard phải có hành vi gì?

10. Flash message giúp tránh “kẹt trạng thái” như thế nào?

- Vì sao flash nên hiện một lần rồi biến mất?
- Nếu dùng session thường để lưu success/error nhưng không xóa, UI sẽ bị lỗi gì?
- Trong bài của em, success/error được set ở đâu và đọc ở đâu?

11. Remember me có rủi ro gì?

- Vì sao không được lưu password trong cookie?
- Nếu muốn làm remember me thật, cần token riêng, token hash ở server và cơ chế thu hồi như thế nào?
- Trong Lab04, checkbox remember me chỉ dùng để thảo luận hay bắt buộc triển khai?

12. Nếu hệ thống mở rộng thành dự án thật, nên cải tiến phần nào?

- Có nên chuyển storage JSON sang database không? Vì sao?
- Cần thêm CSRF token ở những form nào?
- Cần thêm logging, middleware, role-based access hay validation nâng cao không?
- Liên hệ với các tuần sau: database/PDO/CRUD/MVC sẽ giúp cải tiến phần nào?

Câu 3. Yêu cầu nộp bài

Sinh viên nộp một file PDF báo cáo kết quả, không cần gửi toàn bộ source code trong email nếu đã có GitHub link. Báo cáo phải có ảnh chụp màn hình và diễn giải ngắn gọn.

- Tên file PDF: MSSV_HoTenSinhVien_PHP_Lab04.pdf
- Tiêu đề email hoặc tên bài nộp LMS: MSSV_HoTenSinhVien_PHP_Lab04
- Nội dung báo cáo: link GitHub, mô tả bài toán đã chọn, các route đã làm, ảnh chụp giao diện, ảnh chụp test nhỏ T01-T16, trả lời Problem Solving.

- Ghi rõ đã làm đầy đủ hay còn thiếu chức năng nào.
- Ghi rõ phần làm thêm nếu có để được xem xét cộng điểm.
- Deadline và nơi nộp: 15/06/2026 qua email tuantran261083course@gmail.com

Câu 4. Rubric gợi ý chấm điểm

Hạng mục	Điểm	Tiêu chí
Cấu trúc project + Router/Controller	15	Có Front Controller, Router, controller tách nhóm, 404/405 rõ ràng.
Form + input safety + validation	20	Đọc input an toàn, validate đủ rule, lỗi đúng field, giữ old input.
PRG + flash + storage JSON	15	POST thành công redirect GET, refresh không tạo trùng, flash hiện đúng một lần, lưu dữ liệu đúng.
Anti-spam cơ bản	10	Có honeypot và rate limit, test được hành vi bị chặn.
Login/logout/session	20	Cookie flags, regenerate sau login, dashboard protected, timeout, logout sạch, POST logout.
Bài kiểm tra nhỏ T01-T16	10	Có ảnh/minh chứng và mô tả kết quả từng mục kiểm tra.
Problem Solving	10	Trả lời có lập luận, gắn với bài làm, không chép lý thuyết chung chung.
Tổng cộng	100	

Điểm cộng khuyến khích

- Có README hướng dẫn chạy project rõ ràng.
- Có file sample JSON hoặc dữ liệu mẫu đẹp, không lỗi tiếng Việt.
- Có video ngắn hoặc ảnh minh họa flow PRG/flash/session.
- Có kiểm tra bằng curl/Postman ngoài browser.
- Có giải thích rủi ro bảo mật bằng ví dụ thực tế của chính bài làm.
- Có giao diện responsive, dễ nhìn và nhất quán.